

MINOR PROJECT SYNOPSIS

On

“Machine Learning-Based Portfolio Optimization for Risk-Aware Investment Decision Making”

Submitted in partial fulfilment of the requirements for the completion
of minor project [Project/Course Code]

Submitted By:

Adi Aditya Singh ,12419051723, IIOT-B2

Under the supervision of

Dr. Ruchika Sehgal



University School of Automation and Robotics

GURU GOBIND SINGH INDRAPRASTHA UNIVERSITY

East Delhi Campus, Surajmal Vihar, New Delhi-110032

Introduction

Splitting money across different stocks the “right” way is one of the oldest problems in finance, and honestly still one of the hardest to pin down. Markowitz's Mean-Variance framework, part of what we now call Modern Portfolio Theory, gave investors a mathematical way to balance return against risk using historical averages of returns, variances and covariances. The catch is that historical averages don't hold up well as a guide to the future. Shift the time window you're measuring over even slightly and the numbers move around a lot, so a portfolio that looks optimal on paper can behave quite differently once markets actually move.

A common workaround in ML-driven finance is to just predict stock prices or returns directly. But prediction accuracy and portfolio performance turn out to be two different things. A model can forecast returns reasonably well and still hand you a worse portfolio once risk, diversification and turnover enter the picture – and that's easy to miss if a model is judged only on RMSE or R^2 and never actually plugged into a portfolio to see what happens.

This project starts from a different place. Instead of treating price prediction as the end goal, it treats machine learning as just one piece of a bigger quantitative pipeline. Historical market data gets turned into features – returns, volatility, momentum, technical indicators – and models like Linear Regression, Random Forest and XGBoost are trained on those features to estimate expected future returns and/or volatility. Those estimates then go into a portfolio optimizer that decides how to split weights across assets, and the resulting portfolio gets tested out-of-sample through walk-forward backtesting against four standard benchmarks: Equal Weight, Minimum Variance, Maximum Sharpe and Risk Parity.

One thing mean-variance optimization is notoriously bad at is handling noisy inputs gracefully, so a Monte Carlo resampling layer is added around the optimizer to soften that. Rather than trusting one single ML-derived estimate of expected return and covariance, the idea is to simulate many plausible variations of that estimate and re-run the optimizer on each one. That way the final portfolio reflects a spread of plausible futures instead of betting everything on one potentially noisy prediction. The same Monte Carlo setup also comes in handy for computing VaR and CVaR empirically, and for stress-testing the strategy across simulated market conditions later on.

The bigger idea behind structuring things this way is that the ML component gets judged on what actually matters to an investor – Sharpe Ratio, Sortino Ratio, drawdown, CAGR – rather than on how neatly it fits historical price movements. Just as importantly, care is taken to avoid the usual traps that show up in financial ML – look-ahead bias and data leakage in particular – since a model that accidentally peeks into the future will look impressively good in a backtest and then fall apart the moment it's used in real trading.

Literature Survey

1. Markowitz's Mean-Variance framework, and Modern Portfolio Theory more broadly, is still the default starting point for building diversified portfolios based on expected return, variance and covariance.
2. A well-documented weakness of mean-variance optimization is how sensitive it is to estimation error in expected returns – nudge the input estimates even slightly and the resulting portfolio can swing quite a bit. Resampling-based approaches such as the Michaud resampled efficient frontier were proposed specifically to dampen this by averaging optimized weights across many simulated input scenarios.
3. Regression models, tree ensembles like Random Forest and XGBoost, and deep learning approaches such as LSTM/GRU have all shown up in the literature for forecasting asset returns or volatility from historical market data.
4. Minimum Variance, Maximum Sharpe and Risk Parity portfolios are the go-to quantitative benchmarks whenever a new allocation approach is being tested against something established.
5. Financial time-series work generally calls for walk-forward or time-ordered validation instead of random train/test splits – shuffle the data randomly and you risk leaking future information into training, which quietly inflates results.
6. Sharpe Ratio, Sortino Ratio, Maximum Drawdown, Calmar Ratio, VaR and CVaR are the usual suspects for judging portfolio performance beyond just raw return. Monte Carlo simulation is a fairly standard way to estimate VaR/CVaR empirically, especially when returns don't behave like a normal distribution (which, in practice, they rarely do).
7. Whether ML-based forecasts genuinely improve portfolio outcomes – rather than just prediction accuracy – compared with these established baselines still isn't fully settled, and it's worth digging into the existing literature more closely before assuming an answer either way.

A proper literature matrix covering somewhere around 15–30 papers will be put together early on, to actually locate where the gap is instead of assuming one going in.

Problem Statement

Traditional portfolio construction leans on historical return and covariance estimates, and those tend to be noisy and unstable in practice. Meanwhile, a lot of ML-based forecasting work stops right at prediction accuracy and never actually checks whether the improved forecasts translate into a better portfolio – and financial ML studies are notoriously easy to get wrong through look-ahead bias, data leakage, or unknowingly tuning against the test set.

With that in mind, this minor project sets out to build a system that:

- (a) Combines ML-based return/volatility estimation with a quantitative portfolio optimizer.
- (b) Collects historical market data and builds the relevant quantitative and technical features from it.
- (c) Trains ML models – Linear Regression, Random Forest, XGBoost – using only the information that would actually have been available at each decision date.
- (d) Applies Monte Carlo resampling around the ML-derived estimates and reoptimizes across simulated scenarios, so the final asset weights aren't overly sensitive to estimation error in any one ML forecast.
- (e) Feeds these ML-derived (and Monte Carlo-stabilized) estimates into the portfolio optimization step to compute asset weights.
- (f) Backtests the resulting portfolio through a walk-forward, out-of-sample setup that avoids look-ahead bias.
- (g) Compares the results against Equal Weight, Minimum Variance, Maximum Sharpe and Risk Parity portfolios.
- (h) Evaluates everything using Sharpe, Sortino, Maximum Drawdown, CAGR, Calmar Ratio, and Monte Carlo-based VaR/CVaR.
- (i) Runs a handful of robustness checks – including Monte Carlo stress simulations – across different market periods, volatility regimes and transaction cost assumptions.
- (j) Writes everything up in a form that could realistically be extended into a research paper later on.

Proposed Solution

The proposed solution is basically a pipeline: start with raw market data, end with a set of portfolio weights that can be backtested and compared against standard strategies. Broadly, it looks like this:

Market Data → Feature Engineering → ML Model (expected return/volatility) → Monte Carlo Resampling → Portfolio Optimizer → Portfolio Weights → WalkForward Backtest → Performance Evaluation.

Worth noting: the ML model doesn't replace the optimizer, it just feeds it better inputs than a plain historical average would. So rather than the optimizer working off last year's average return, it's working off an ML-generated estimate that (ideally) reacts a little more sensibly to what's happened recently in the market.

To deal with how sensitive mean-variance optimization is to estimation error (see point 2 in the Literature Survey), Monte Carlo resampling sits between the ML estimation step and the optimizer. Instead of feeding one single ML point estimate of expected return/covariance straight into the optimizer, many synthetic variations of that estimate get simulated – using the ML model's own residual/error distribution as the source of noise – and the optimizer is re-run for each one. The resulting weight vectors are then averaged (or their median is taken) to get the final, hopefully more stable, allocation. This same Monte Carlo machinery gets reused twice more later in the pipeline: once to estimate VaR and CVaR empirically from simulated portfolio return distributions instead of assuming a closed-form normal formula, and again to generate synthetic market paths – via bootstrap or block-bootstrap resampling of historical returns – for the robustness-check stage, so performance ends up being judged across many plausible futures rather than just the one historical path we happen to have.

The exact optimizer formulation – maximizing $(w^T \hat{\mu} - r_f) / \sqrt{w^T \Sigma w}$ subject to $\Sigma w = 1$ and per-asset weight caps, roughly – may still get adjusted once the literature review wraps up and the final methodology is locked in.

Workflow Diagram

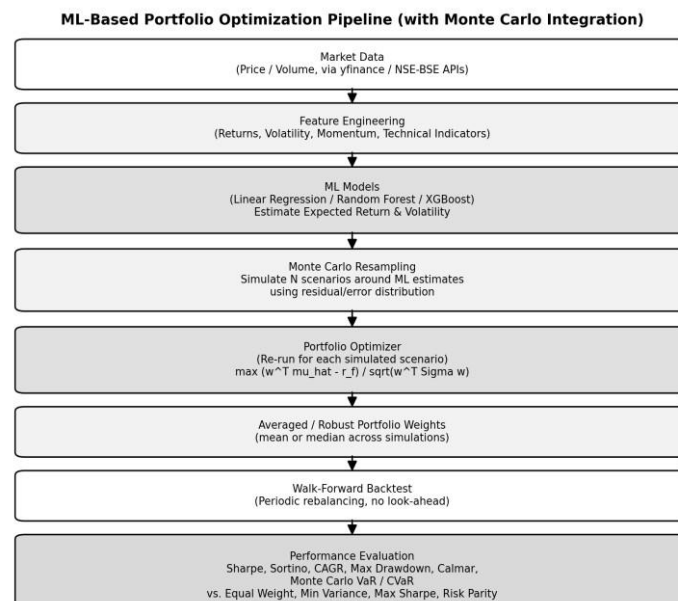


Figure 1: End-to-end ML-based portfolio optimization pipeline with Monte Carlo resampling integrated between the ML estimation and optimization stages.

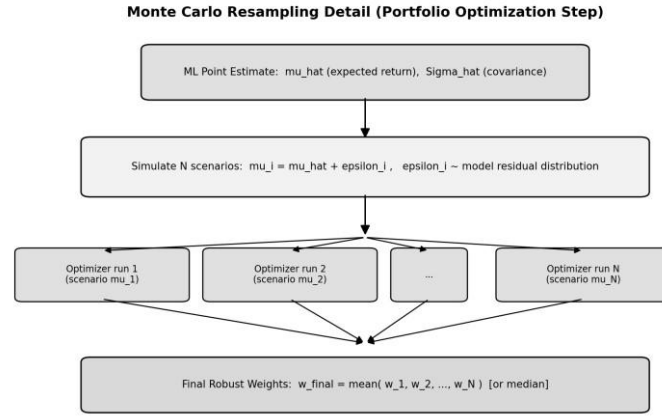


Figure 2: Detail of the Monte Carlo resampling step - simulating scenarios around the ML point estimate, re-optimizing on each, and averaging the resulting weights.

Below are the core formulae behind the ML models, the portfolio optimizer, the Monte Carlo resampling step, and the evaluation metrics used throughout. The exact functional forms might get refined a bit once the literature review and dataset are finalized.

1. XGBoost (Gradient Boosted Trees)

$$\hat{y}_i^{(t)} = \sum_{k=1}^t f_k(x_i)$$

Trained by additively minimizing a regularized objective at each boosting round t :

$$L^{(t)} = \sum_i l(y_i, \hat{y}_i^{(t-1)} + f^t(x_i)) + \sum_k \Omega(f_k)$$

$$\Omega(f) = \gamma T + (1/2)\lambda \|w\|^2$$

where l is a differentiable loss function (e.g., squared error), T is the number of leaves, w the leaf weights, and γ, λ are regularization parameters controlling tree complexity.

2. Portfolio Optimizer (Maximum Sharpe / Mean-Variance)

$$\max_w (w^T \hat{\mu} - r_f) / \sqrt{w^T \hat{\Sigma} w}$$

subject to:

$$\sum_i w_i = 1, \quad 0 \leq w_i \leq w_{max}$$

where $\hat{\mu}$ is the (ML-estimated) expected return vector, $\hat{\Sigma}$ the covariance matrix, r_f the risk-free rate, and w the vector of portfolio weights.

3. Minimum Variance Portfolio $\min_w w^T \hat{\Sigma} w$ subject to

$$\sum_i w_i = 1$$

4. Risk Parity Portfolio $w_i \cdot (\hat{\Sigma} w)_i = w_j \cdot (\hat{\Sigma} w)_j$ for all i, j

i.e., asset weights are chosen so that each asset contributes equally to total portfolio risk.

5. Monte Carlo Resampling of Portfolio Weights $\mu_i = \hat{\mu} + \varepsilon_i$, $\varepsilon_i \sim$ (empirical or parametric) residual distribution, $i = 1, \dots, N$ $w_i = \text{Optimizer}(\mu_i, \hat{\Sigma})$

$$w_{final} = (1/N) \sum_{i=1}^N w_i \quad (\text{or } \text{median}_i w_i)$$

where N (e.g., 500-1000) simulated scenarios are drawn around the ML point estimate and the optimizer is re-solved for each, so the final weights are averaged across plausible variations in the return estimate rather than relying on a single potentially noisy forecast.

6. Performance Evaluation Metrics

Sharpe Ratio:

$$\text{Sharpe} = (R_p - R_f) / \sigma_p$$

Sortino Ratio (uses downside deviation σ_d instead of total volatility): Sortino =

$$(R_p - R_f) / \sigma_d$$

Compound Annual Growth Rate:

$$\text{CAGR} = (V_{\text{end}} / V_{\text{start}})^{(1/n)} - 1$$

Maximum Drawdown:

$$\text{MDD} = \min_t [(V_t - \max_{s \leq t} V_s) / \max_{s \leq t} V_s]$$

Calmar Ratio:

$$\text{Calmar} = \text{CAGR} / |\text{MDD}|$$

7. Monte Carlo Value-at-Risk (VaR) and Conditional VaR (CVaR)

Given N simulated portfolio return paths $\{R_1, R_2, \dots, R_n\}$ (drawn from the fitted return distribution or bootstrapped from historical residuals):

$$\text{VaR}_\alpha = -\text{Quantile}_\alpha(\{R_1, \dots, R_n\})$$

$$\text{CVaR}_\alpha = -E[R / R \leq -\text{VaR}_\alpha]$$

VaR_α is the loss threshold not expected to be exceeded with probability $(1-\alpha)$, while CVaR_α (also called Expected Shortfall) captures the average loss in the tail beyond that threshold - giving a fuller picture of downside risk than VaR alone.

Methodology

At a high level, the work breaks down into these stages:

1. Collect historical price/volume data for a defined stock universe.
2. Engineer features – returns, rolling volatility, momentum and technical indicators.
3. Train and validate ML models to estimate expected return and/or volatility, using strictly only past information at each point in time.
4. Apply Monte Carlo resampling around the ML estimates: simulate N synthetic scenarios of expected return/covariance from the model's residual distribution, then re-run the portfolio optimizer on each simulated scenario.

5. Average out (or take the median of) the resulting weight vectors across simulations to land on a single, more robust set of asset weights for that rebalance date.
6. Run a walk-forward backtest, rebalancing periodically instead of testing on one static split, repeating steps 3–5 at each rebalance date.
7. Evaluate the resulting portfolio against the four baseline strategies using risk-adjusted metrics, computing VaR and CVaR empirically via Monte Carlo simulation of the portfolio's return distribution.
8. Carry out robustness checks by generating synthetic market paths through bootstrap/block-bootstrap resampling of historical returns, then re-running the full comparison (proposed strategy vs. all four benchmarks) across these simulated paths to get a distribution of outcomes rather than one single-point result.

Motivation

There's a real gap between a model that predicts returns well on paper and one that actually helps an investor make better decisions. Classical optimization is limited by how noisy its return estimates tend to be, and machine learning is only genuinely worth using here if it improves real portfolio outcomes rather than just fitting historical patterns a bit more closely.

A lot of existing projects either stop at price prediction without ever building an actual portfolio around it, or they apply ML without being careful about validation – which tends to make results look better than they really are. Monte Carlo resampling gets added here specifically to keep the mean-variance step honest about estimation uncertainty, rather than treating any single ML forecast as gospel. That's really the gap this project is trying to close: keep the ML honest by testing it inside a full portfolio pipeline, evaluated walk-forward, against benchmarks that are already well established in quantitative finance. Done carefully enough, this same setup could probably form the basis of a research paper down the line too.

GANTT Chart (16-Week Timeline)

Week	Work
W1–W2	Literature review, research question and dataset finalization
W3–W4	Data collection, cleaning and feature engineering
W5–W6	Implementation of quantitative baselines (Equal Weight, Min Variance, Max Sharpe, Risk Parity)

W7–W9	ML model development for return/volatility estimation
W10–W11	Integration of ML estimates into portfolio optimization, with Monte Carlo resampling for robust weight estimation
W12–W13	Walk-forward backtesting, performance evaluation and Monte Carlo VaR/CVaR estimation
W14	Robustness checks across periods, regimes and transaction costs, incl. Monte Carlo stress simulations
W15	Result compilation and comparative analysis
W16	Documentation and final submission

Hardware Requirements

- Development Laptop/PC - Intel Core i5/AMD Ryzen 5 or above, 16 GB RAM, 256 GB+ SSD
- (Optional) GPU-enabled/cloud instance - for deep learning models, if used
- Reliable Internet Connection - for market data acquisition via APIs

Software Requirements

- Programming Language: Python (Pandas, NumPy)
- Machine Learning: scikit-learn, XGBoost (TensorFlow/PyTorch if LSTM/GRU used)
- Quantitative Finance: PyPortfolioOpt / cvxpy, statsmodels
- Monte Carlo & Statistical Resampling: NumPy/SciPy random sampling, arch (bootstrap), joblib (parallel simulation runs)
- Backtesting: backtrader / vectorbt / custom walk-forward framework
- Market Data: Yahoo Finance (yfinance) / NSE-BSE / other financial APIs
- Visualization: Matplotlib, Seaborn
- Tools: Jupyter Notebook / VS Code, Git and GitHub

References

- [1] H. Markowitz, "Portfolio Selection," *The Journal of Finance*, vol. 7, no. 1, pp. 77–91, 1952.
- [2] R. O. Michaud, *Efficient Asset Management: A Practical Guide to Stock Portfolio Optimization and Asset Allocation*. Boston, MA: Harvard Business School Press, 1998.
- [3] W. F. Sharpe, "The Sharpe Ratio," *The Journal of Portfolio Management*, vol. 21, no. 1, pp. 49–58, 1994.
- [4] R. T. Rockafellar and S. Uryasev, "Optimization of Conditional Value-at-Risk," *Journal of Risk*, vol. 2, no. 3, pp. 21–41, 2000.
- [5] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proc. 22nd ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
- [6] B. Kelly, S. Malamud, and K. Zhou, "The Virtue of Complexity in Return Prediction," *The Journal of Finance*, vol. 79, no. 1, pp. 459–503, 2024.
- [7] O. Jin and H. El-Saawy, "Portfolio Management using Reinforcement Learning," *Stanford University, CS229 Project Report*, 2016.
- [8] S. Perrin and T. Roncalli, "Machine Learning Optimization Algorithms & Portfolio Allocation," *Amundi Quantitative Research*, arXiv:1909.10233, 2019.
- [9] M. López de Prado, *Advances in Financial Machine Learning*. Hoboken, NJ: John Wiley & Sons, 2018.
- [10] AQR Capital Management, "Can Machines Build Better Stock Portfolios?" *Alternative Thinking Series*, 2024.

(Additional papers identified during the full literature review will be incorporated here, expanding this list to the 15–30 sources referenced in the Literature Survey.)